LeetCode – Nov 2026 | Week 1

Author: Eric Zhou
Email: eric_ds_zhou@outlook.com
Date: Nov 6, 2026

Introduction

This week, I am starting a new series focused on solving LeetCode problems using `Java`. In this series, I will document my problem-solving process, explain the algorithms I use, and analyze their time and space complexities.

Leetcode 455 - Assigned Cookies

Question

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child `i` has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie `j` has a size s[j]. If s[j] >= g[i], we can assign the cookie `j` to the child `i`, and the child `i` will be content. Your goal is to maximize the number of your content children and output the maximum number.

Intuition

The goal of this problem is to maximize the number of content children by assigning each child at most one cookie.

Initially, we might wonder whether assigning a large cookie to a child with a small greed factor would waste the cookie.

However, our goal is not to minimize the size of unused cookies, but to maximize the number of satisfied children.

Therefore, we can use a `Greedy Algorithm` to solve this problem.

The main idea is to satisfy the child with the smallest greed factor first, using the smallest available cookie that can satisfy that child.

Approach

First, we sort both arrays in ascending order using Arrays.sort().

`g` stores the greed factors of the children.

`s` stores the sizes of the cookies.

Then, we use two pointers, `i` and `j`, both initialized to 0.

`i` represents the current child and the number of satisfied children.

`j` represents the current cookie.

We iterate through both arrays using a while loop.

For each iteration, we check:

g[i] <= s[j]

If this condition is `true`, the current cookie can satisfy the child. Therefore, we increment both `i` and `j`.

Otherwise, the cookie is too small. Since both arrays are sorted, this cookie cannot satisfy any remaining child, so we only increment `j`.

We repeat this process until we have checked all children or all cookies.

Finally, we return `i`, which represents the maximum number of satisfied children.

Time Complexity:

$$\Theta(n \log n + m \log m)$$

Let n be the number of children and m be the number of cookies.

Sorting both arrays takes:

$$O(n \log n + m \log m)$$

The two-pointer traversal takes:

$$O(n + m)$$

Therefore, the overall time complexity is:

$$\boxed{\Theta(n \log n + m \log m)}$$

assuming the standard comparison-based sorting behavior of Arrays.sort() for integer arrays.

Code

```java
import java.util.Arrays;

class Solution {
    public int findContentChildren(int[] g, int[] s) {
        Arrays.sort(g);
        Arrays.sort(s);

        int i = 0;
        int j = 0;

        while (i < g.length && j < s.length) {
            if (g[i] <= s[j]) {
                i++;
            }
```

```
                j++;
            }

            return i;
        }
    }
```